Scikit Learn Implementation Using Quadratic Equation

.

Step 1: Installing Scikit-Learn

First we need to install our dependency, the Python library scikit-learn (siklearn) which is a free software machine learning library for the Python Programming Language. The random library is part of the python standard library so we don't need to install it, it's already available for us !

pip install scikit-learn

```
C:\Users\Admin\Documents\pyCodes>pip install scikit-learn
Collecting scikit-learn
  Downloading scikit_learn-1.8.0-cp314-cp314-win_amd64.whl.metadata (11 kB)
Requirement already satisfied: numpy>=1.24.1 in c:\users\admin\appdata\local\python\pythoncore-3.14-64\lib\site-packages
-learn) (2.4.4)
Collecting scipy>=1.10.0 (from scikit-learn)
  Downloading scipy-1.17.1-cp314-cp314-win_amd64.whl.metadata (60 kB)
Collecting joblib>=1.3.0 (from scikit-learn)
  Downloading joblib-1.5.3-py3-none-any.whl.metadata (5.5 kB)
Collecting threadpoolctl>=3.2.0 (from scikit-learn)
  Downloading threadpoolctl-3.6.0-py3-none-any.whl.metadata (13 kB)
Downloading scikit_learn-1.8.0-cp314-cp314-win_amd64.whl (8.1 MB)
   ---------------------------------------- 8.1/8.1 MB 5.7 MB/s  0:00:01
Downloading joblib-1.5.3-py3-none-any.whl (309 kB)
Downloading scipy-1.17.1-cp314-cp314-win_amd64.whl (37.3 MB)
   ---------------------------------------- 37.3/37.3 MB 4.9 MB/s  0:00:07
Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)
Installing collected packages: threadpoolctl, scipy, joblib, scikit-learn
Successfully installed joblib-1.5.3 scikit-learn-1.8.0 scipy-1.17.1 threadpoolctl-3.6.0

[notice] A new release of pip is available: 25.3 => 26.1
[notice] To update, run: C:\Users\Admin\AppData\Local\Python\pythoncore-3.14-64\python.exe -m pip install --upgrade pip
```

Step 2: Create Your Training Dataset

Usually if you have a data set, you would want to split it into a training set, validation set, and testing set. Since this is a simple example we are creating our own training set to train our linear regression model with. The training set will contain the input and expected output values of the data set that I want to train the model on. The input will be random integer values created by the library Random we will use the method randint(a,b) , which returns a random integer N such that a <= N <= b. The output will be determined by a function that we will create.

Recall a function usually called 'f' takes in some input/parameter value usually called 'x' and produces some output value usually represented by 'y', such that f(x) = y. The function that I will create will take in multiple parameters so not just 'x' but 'a', 'b', 'c' , 'd' instead such that we get the following function.

f(a, b, c, d) = 7a + 3b + 4c + 9d = y

Example of how this function works, if we had values as a = 1, b = 2, c = 3 and d = 4 then y would equal **61** like the following:

$$f(a, b, c, d) = 7(1) + 3(2) + 4(3) + 9(4) = 61$$

Example of Training Set:

Input				Output
a	b	c	d	Y
1	2	3	4	**61**
2	3	5	1	**52**
10	20	30	40	**610**
0	1	3	5	**60**

An Example of Training Set

Once we have created the training set, we will split each row into two lists (an input training set, and its corresponding output training set). So now we have two lists of all the inputs and their corresponding outputs.

Code to Generate Training Set:

```
'''
#Performing Linear Regression Algorithm for the quadratic equation
#X = 7a + 3b + 4c + 9d
#Using sklearn module to perform the linear regression equation.
'''
#Importing required modules
from random import randint
from sklearn.linear_model import LinearRegression

#Constructing training dataset using Random module

#Setting up the limit and counts to generate the data
train_set_limit = 1000
train_set_count = 150

#generated trained data will be stored as list in train_input variable
train_input = list()

#generated outputs are stored here. It is learned data.
train_output = list()

#it will assign different unique values to the variables until the count is met
for i in range(train_set_count):
    a = randint(0, train_set_limit)
    b = randint(0, train_set_limit)
    c = randint(0, train_set_limit)
    d = randint(0, train_set_limit)
    op = (7*a)+(3*b)+(4*c)+(9*d)
    train_input.append([a,b,c,d])
    train_output.append(op)
```

Step 3: Train Our Linear Regression Model

Next we will use the method LinearRegression() from the Python library scikit-learn to train and create our model. This model will try to “model” the function that we created for the training dataset.

Note: When we ‘fit’ a function that is just another word for training.

```
#Training the model
predictor = LinearRegression(n_jobs = -1)
predictor.fit(X = train_input, y = train_output)
```

Step 4: Test Our Linear Regression Model

Now let's see if our Regression function can approximate the function that we created and give us an accurate prediction (the correct answer).

X = [[10, 20, 30, 40]]

The outcome should be 7*10 + 3*20 + 4*30 + 9*40 = 610. Let's see what we got…

```
#Testing the model
#Sample test values
x_test = [[10,20,30,40]]
outcome = predictor.predict(X = x_test)
coefficients = predictor.coef_
```

```
#Printing the prediction
print("Outcome of given test data is: ")
print("Outcome: {} \nCoefficients: {}".format(outcome,coefficients))
```

So after executing the above code, finally we will get the result as the below snippet:

```
Outcome of given test data is:
Outcome: [610.]
Coefficients: [7. 3. 4. 9.]
```

The model accessed the training data, which it used to calculate the weights (coefficients) to assign to the inputs to arrive at the correct output. When giving the model test data, it successfully managed to get the correct answer.